5G FUNDAMENTALS AND ARCHITECTURES
EC23712
ASSIGNMENT – 1 (TEAM – 2)

Implement the traffic profile TCP uplink, single flow, 60 s - compare against downlink

1. Objective

The objective of this experiment is to set up the Open5GS 5G core network on Ubuntu and generate TCP uplink traffic using iperf3 for a duration of 60 seconds. The throughput values obtained during the experiment are visualized using Python. The uplink performance is then compared with TCP downlink traffic to understand the variation in throughput and overall network behaviour.

2. Software and Tools Used

Software	Purpose
Ubuntu 22.04	Operating System
Open5GS	5G Core Network
MongoDB	Subscriber Database
Node.js 20	Open5GS WebUI Support
Open5GS WebUI	Subscriber Management
iperf3	TCP Traffic Generation
Python 3	Data Processing
Matplotlib	Graph Plotting

3. System Setup

3.1 Network Architecture

1. Single laptop running full 5G core
2. Control Plane: NRF, SCP, AMF, SMF, AUSF, UDM, PCF, NSSF, BSF, UDR
3. User Plane: UPF
4. TUN interface: ogstun (10.45.0.1/16)

3.2 Steps Performed

The experiment was carried out on a single Ubuntu machine where the complete Open5GS core network was running.

The setup included:

- Open5GS Control Plane Functions
- Open5GS User Plane Function (UPF)
- MongoDB Subscriber Database
- Open5GS WebUI
- TUN Interface (ogstun)
- iperf3 for traffic generation
- Python with Matplotlib for visualization

4. Procedure

The following steps were performed during the experiment:

1. Installed MongoDB and configured it with Open5GS.
2. Built Open5GS from source.
3. Created and configured the **ogstun** interface.
4. Started all Open5GS core network functions.
5. Started the iperf3 server.
6. Generated TCP uplink traffic for 60 seconds.
7. Generated TCP downlink traffic for 60 seconds.
8. Saved both outputs in JSON format.
9. Used Python and Matplotlib to plot throughput graphs.
10. Compared uplink and downlink throughput characteristics.

5. Commands Used

i. Start iperf3 Server

```
iperf3 -s
```

ii. Generate TCP Uplink Traffic

```
iperf3 -c 127.0.0.1 -t 60 -i 1 --json > uplink_result.json
```

iii. Generate TCP Downlink Traffic

```
iperf3 -c 127.0.0.1 -t 60 -i 1 -R --json > downlink_result.json
```

iv. Generate Graph

```
python3 plot.py
```

6. Results

6.1 TCP Uplink Performance

Parameter	Value
Test Duration	60 seconds
Protocol	TCP
Flow Type	Single Flow
Average Throughput	46,751 Mbps
Peak Throughput	Approximately **57,000 Mbps**
Minimum Throughput	Approximately **34,500 Mbps**

Figure 1

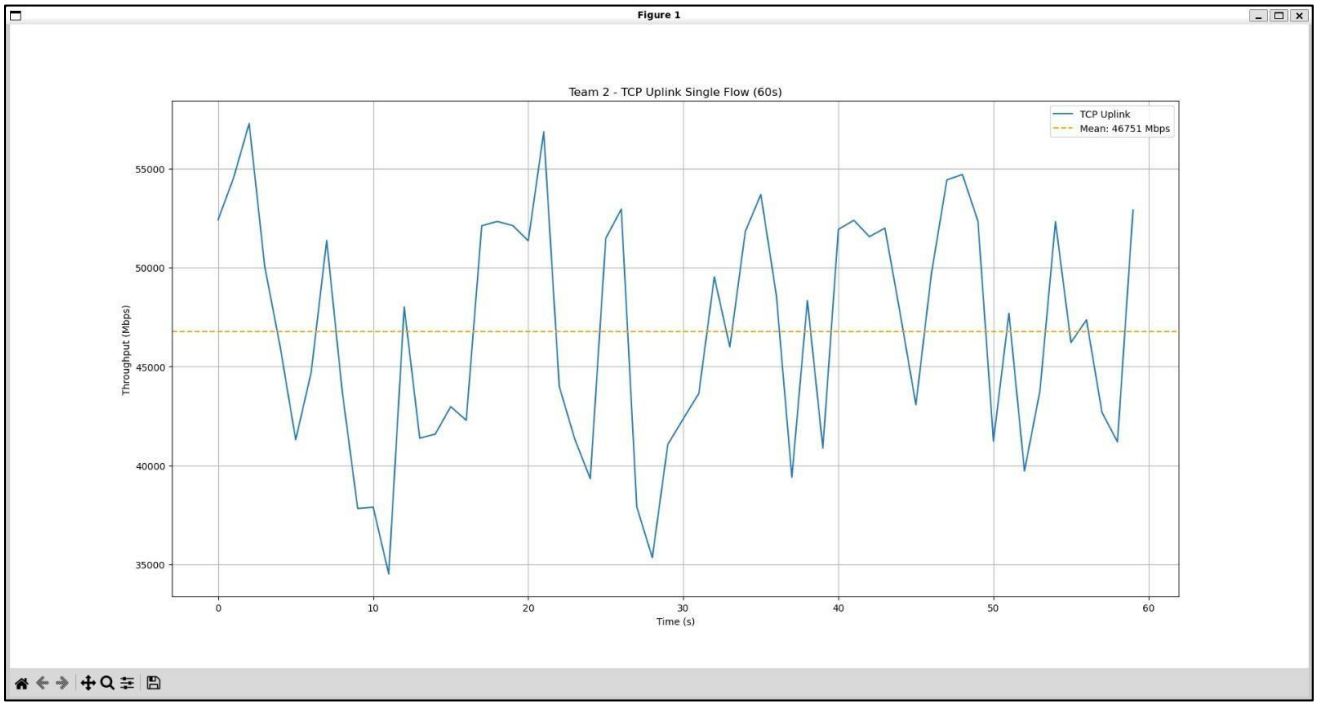

Figure 6.1. TCP Uplink Throughput for Single Flow (60 Seconds)

6.2 Observation

The TCP uplink throughput varies throughout the experiment. The throughput ranges approximately from 34 Gbps to 57 Gbps, while maintaining an average throughput of 46.75 Gbps. These variations occur because TCP continuously adjusts its congestion window according to network conditions.

7. Uplink and Downlink Comparison

Both uplink and downlink traffic were generated for 60 seconds and compared using a combined throughput graph.

Parameter	Uplink	Downlink
Protocol	TCP	TCP
Duration	60 s	60 s
Average Throughput	46,751 Mbps	47,443 Mbps
Flow Type	Single Flow	Single Flow

Figure 2

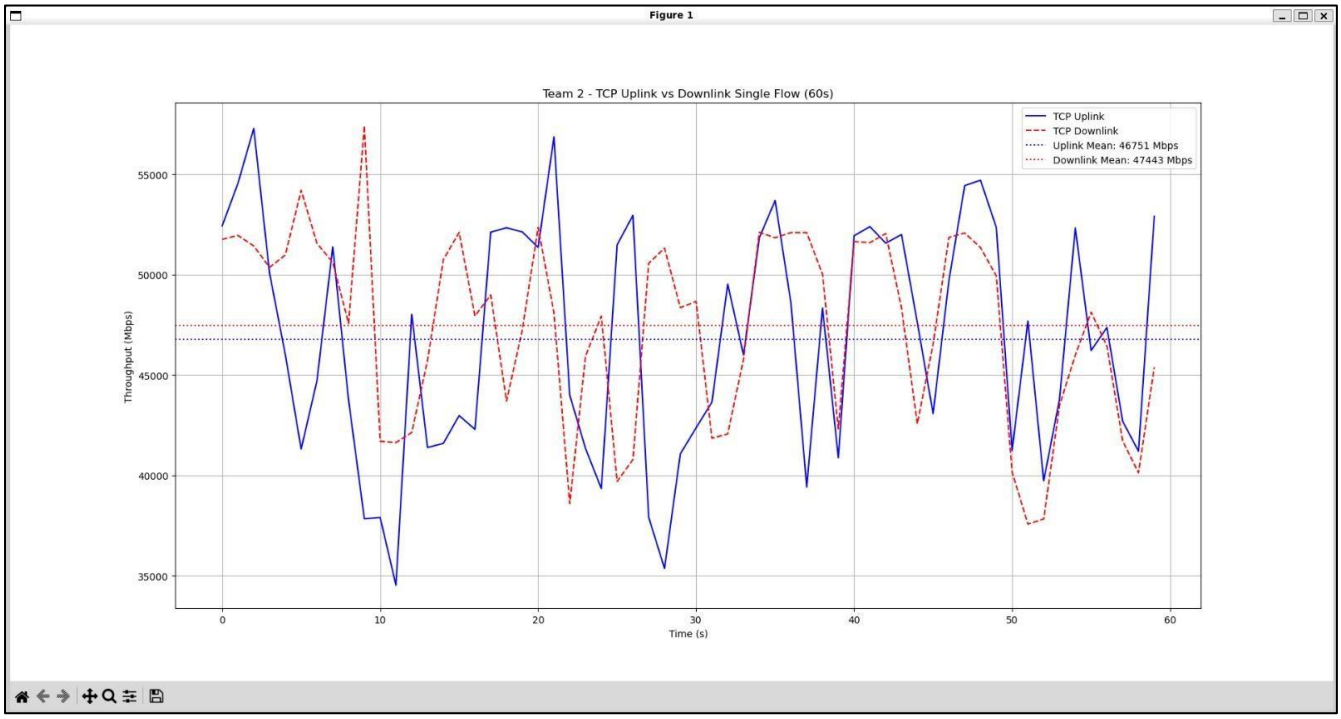

Figure 7.1. Comparison of TCP Uplink and TCP Downlink Throughput

- The comparison graph shows that both TCP uplink and downlink achieve high throughput throughout the experiment. The average throughput of the downlink is slightly higher than that of the uplink.
- The uplink graph shows more fluctuations because the client controls the transmission rate, and TCP congestion control continuously adjusts the sending window based on acknowledgements received from the receiver.
- The downlink traffic appears comparatively smoother because the server side maintains a more consistent transmission rate.

8. Conclusion

The experiment successfully demonstrated TCP uplink traffic generation using Open5GS and iperf3 on a virtualized 5G core network.

The throughput values were collected for 60 seconds and visualized using Python. The average uplink throughput obtained was approximately 46.75 Gbps, while the average downlink throughput was approximately 47.44 Gbps.

The comparison shows that TCP uplink experiences slightly higher fluctuations due to congestion control, whereas downlink remains comparatively stable. Overall, the experiment confirms that Open5GS provides an effective platform for evaluating TCP traffic performance in a software-based 5G network.

9. References

1. Open5GS Documentation
   https://open5gs.org/open5gs/docs/guide/02-building-open5gs-from-sources/
2. iperf3 Documentation
   https://iperf.fr/
3. Open5GS GitHub Repository
   https://github.com/open5gs/open5gs